Python Assignment
Topic-10: Advanced Python (map(), reduce(), filter(), Closures and Decorators)

Q1. How functional programming works in Python.

Ans.

Functional programming is a style of programming where functions are used as the main building blocks. In Python, functions can be treated like variables and can be passed as arguments. It focuses on writing clean and reusable code. Functional programming avoids changing data directly and instead creates new data. It makes programs easier to understand and reduces errors. Python supports functional programming along with other styles like object-oriented programming. It is useful for writing short and efficient code.

Example:

```
def add(a, b):
   return a + b

print(add(2, 3))
```

Q2. Using map(), reduce(), and filter() functions for processing data.

Ans.

Python provides built-in functions like map(), reduce(), and filter() to process data easily. The map() function is used to apply a function to all elements of a list. The filter() function is used to select elements based on a condition. The reduce() function is used to combine all elements into a single value. These functions help in writing short and efficient code. They are commonly used in data processing tasks.

Example:

```
from functools import reduce

nums = [1,2,3,4]
print(list(map(lambda x: x*2, nums)))
print(list(filter(lambda x: x%2==0, nums)))
print(reduce(lambda a,b: a+b, nums))
```

Q3. Introduction to closures and decorators.

Ans.

Closures are functions inside another function that remember the outer function variables. They allow us to use data even after the outer function has finished execution. Decorators are used to modify or extend the behavior of functions. They are written using the @ symbol. Decorators help in adding extra features like logging or checking conditions without changing the original function. Both closures and decorators are advanced concepts but very useful in real programs.

Example:

```
def outer():
   x = 10
   def inner():
      print(x)
   return inner

f = outer()
f()
```